

Modul Praktikum Flutter Lanjutan

Pertemuan 11: Integrasi Flutter dengan API (Repository Pattern)

Durasi	Level	Bentuk kerja	Prasyarat
150 menit	Lanjut	Individu	Pertemuan 10 (login JWT, KegiatanApi, LaporanApi)

1. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Menerapkan **repository pattern** untuk memisahkan UI dari sumber data.
2. Menukar sumber data (SQLite ↔ API) tanpa mengubah kode halaman.
3. Menampilkan status *loading*, galat, dan tombol coba lagi untuk data jaringan, serta *pull-to-refresh*.
4. Mengirim beberapa request secara paralel dengan `Future.wait`.
5. Menguji widget dengan repository palsu (*fake*) tanpa server maupun database.

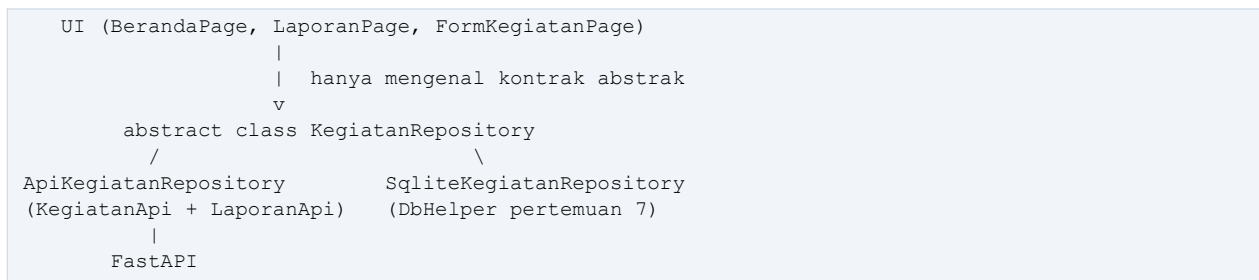
2. Alat dan Bahan

- Hasil pertemuan 10 (atau snapshot `pert10/tugas10`). Stack Docker berjalan dan sudah ada akun uji.
- Emulator atau HP yang dapat menjangkau API (cek di halaman Status Server).

3. Teori Singkat

Saat ini halaman-halaman `TabungKu` memanggil `DbHelper` (SQLite) secara langsung. Mengganti setiap pemanggilan menjadi `KegiatanApi` di semua halaman akan menyebarkan detail HTTP ke seluruh UI.

Repository pattern menambahkan satu lapisan kontrak:



Keuntungan	Contoh di TabungKu
Sumber data dapat ditukar	--dart-define=MODE_DATA=lokal kembali ke SQLite
UI mudah diuji	<code>FakeRepository</code> di test, tanpa server
Detail tersembunyi	UI tidak tahu soal URL, JSON, maupun header token
Optimasi di satu tempat	Laporan 7 hari: SQLite 7 query, API 1 request <code>/laporan/rentang</code>

Data jaringan bisa gagal. Berbeda dengan SQLite, request ke server dapat lambat, *timeout*, atau ditolak. Setiap halaman harus menangani tiga keadaan (pola pertemuan 4): memuat, galat (dengan tombol coba lagi), dan data. Operasi tulis yang gagal tidak boleh menutup form, agar isian pengguna tidak hilang.

4. Langkah Praktikum

Bagian A: Kontrak Repository

Pindahkan `versiData` dari `db_helper.dart` ke berkas baru

`lib/data/kegiatan_repository.dart`, lalu hapus `_berubah()` dari `DbHelper`. Kenaikan versi ini menjadi tugas repository:

```
/// Naik setiap kali data berubah. Halaman yang menampilkan data
/// mendengarkan notifier ini lalu memuat ulang datanya.
final versiData = ValueNotifier<int>(0);

/// Kontrak sumber data kegiatan. Halaman hanya mengenal kelas abstrak ini,
/// sehingga sumber data (SQLite atau API) dapat ditukar tanpa mengubah UI.
abstract class KegiatanRepository {
  Future<List<Kegiatan>> pada(String tanggal);
  Future<Ringkasan> ringkasan(String tanggal);
  Future<LaporanHarian> laporanHarian(String tanggal);
  Future<List<MapEntry<DateTime, Ringkasan>>> rentang(
    DateTime mulai,
    DateTime sampai,
  );

  Future<void> tambah(Kegiatan k);
  Future<void> ubah(Kegiatan k);
  Future<void> setSelesai(Kegiatan k, bool selesai);
  Future<void> hapus(int id);
}

/// Repository yang sedang dipakai aplikasi. Test dapat menggantinya
/// dengan repository palsu.
KegiatanRepository repo = modeData == 'lokal'
  ? SQLiteKegiatanRepository()
  : ApiKegiatanRepository();
```

Tambahkan di `lib/config/api_config.dart`:

```
/// Sumber data kegiatan: 'api' (bawaan) atau 'lokal' (SQLite seperti
/// pertemuan 7). Contoh: flutter run --dart-define=MODE_DATA=lokal
const modeData = String.fromEnvironment('MODE_DATA', defaultValue: 'api');
```

Bagian B: Dua Implementasi

Implementasi API membungkus kelas dari pertemuan 9–10:

```
class ApiKegiatanRepository implements KegiatanRepository {
  final KegiatanApi _kegiatan;
  final LaporanApi _laporan;

  ApiKegiatanRepository({KegiatanApi? kegiatan, LaporanApi? laporan})
    : _kegiatan = kegiatan ?? KegiatanApi(),
      _laporan = laporan ?? LaporanApi();

  @override
  Future<List<Kegiatan>> pada(String tanggal) =>
    _kegiatan.daftar(tanggal: tanggal);

  @override
  Future<Ringkasan> ringkasan(String tanggal) async =>
    (await _laporan.harian(tanggal)).ringkasan;

  @override
  Future<LaporanHarian> laporanHarian(String tanggal) =>
    _laporan.harian(tanggal);

  /// Satu request untuk seluruh rentang, bukan satu request per hari.
  @override
  Future<List<MapEntry<DateTime, Ringkasan>>> rentang(
    DateTime mulai,
```

```

    DateTime sampai,
  ) => _laporan.rentang(mulai, sampai);

  @override
  Future<void> tambah(Kegiatan k) async {
    await _kegiatan.tambah(k);
    versiData.value++;
  }

  @override
  Future<void> setSelesai(Kegiatan k, bool selesai) async {
    await _kegiatan.setSelesai(k.id!, selesai);
    versiData.value++;
  }

  // ubah() dan hapus() mengikuti pola yang sama.
}

```

SqliteKegiatanRepository membungkus DbHelper dengan cara serupa. laporanHarian menggabungkan DbHelper.ringkasan dan DbHelper.pengeluaranPerKategori, sedangkan rentang memanggil DbHelper.ringkasan untuk setiap hari. Implementasi lengkapnya ada di snapshot.

Checkpoint B: flutter analyze tidak menemukan galat pada kedua kelas, artinya semua method kontrak sudah diimplementasikan.

Bagian C: Komponen Galat yang Dipakai Ulang

lib/widgets/galat_view.dart membedakan galat dari server (ApiException, pesannya ditampilkan) dan galat jaringan (pesan umum):

```

class GalatView extends StatelessWidget {
  final Object galat;
  final VoidCallback onCobaLagi;

  const GalatView({super.key, required this.galat, required this.onCobaLagi});

  @override
  Widget build(BuildContext context) {
    final e = galat;
    final pesan = e is ApiException
      ? e.pesan
      : 'Tidak dapat terhubung ke server.\nPeriksa koneksi dan alamat API.';
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Icon(e is ApiException ? Icons.error_outline : Icons.cloud_off, size: 64),
            const SizedBox(height: 12),
            Text(pesan, textAlign: TextAlign.center),
            const SizedBox(height: 16),
            FilledButton.icon(
              onPressed: onCobaLagi,
              icon: const Icon(Icons.refresh),
              label: const Text('Coba lagi'),
            ),
          ],
        ),
      ),
    );
  }
}

/// Menampilkan pesan galat singkat di Snackbar.
void tampilkanGalat(BuildContext context, Object e) { ... }

```

Bagian D: Halaman Hari Ini

Ganti semua DbHelper di `beranda_page.dart` dengan `repo`. Tiga perubahan penting:

```
Future<DataHari> _ambil() async {
  final tanggal = fmtTanggal(DateTime.now());
  // Dua request dikirim bersamaan, bukan berurutan.
  final hasil = await Future.wait([
    repo.pada(tanggal),
    repo.ringkasan(tanggal),
  ]);
  return DataHari(hasil[0] as List<Kegiatan>, hasil[1] as Ringkasan);
}

/// Untuk RefreshIndicator: indikator berputar sampai data baru tiba.
Future<void> _tarikSegarkan() async {
  _muat();
  try {
    await _future;
  } catch (_) {
    // Galat sudah ditampilkan oleh FutureBuilder.
  }
}

Future<void> _jalankan(Future<void> Function() aksi) async {
  try {
    await aksi();
  } catch (e) {
    if (mounted) tampilkanGalat(context, e);
  }
}
```

Di `build()`: **galat** ditampilkan dengan `GalatView(galat: snapshot.error!, onCobaLagi: _muat).ListView` **dibungkus** `RefreshIndicator(onRefresh: _tarikSegarkan, ...)` dengan `physics: const AlwaysScrollableScrollPhysics()` agar tetap bisa ditarik walau isinya sedikit. **Centang** memakai `_jalankan(() => repo.setSelesai(k, !k.selesai))`, dan **hapus** memakai `_jalankan(() => repo.hapus(k.id!))`.

****Mengapa setSelesai dan bukan ubah?*** Mencentang cukup mengirim `PATCH {"selesai": true}` (beberapa byte) daripada `PUT` seluruh data. Ini lebih hemat dan mengurangi risiko menimpa perubahan dari perangkat lain.

Bagian E: Halaman Laporan dan Form

Laporan: ambil laporan harian dan 7 hari terakhir sekaligus:

```
Future<DataLaporan> _ambil() async {
  final basis = _tanggal;
  final awal = DateTime(basis.year, basis.month, basis.day - 6);
  final hasil = await Future.wait([
    repo.laporanHarian(fmtTanggal(basis)),
    repo.rentang(awal, basis),
  ]);
  final harian = hasil[0] as LaporanHarian;
  final rentang = hasil[1] as List<MapEntry<DateTime, Ringkasan>>;
  // Terbaru di atas, sama seperti pertemuan 7.
  return DataLaporan(harian.ringkasan, harian.perKategori, rentang.reversed.toList());
}
```

Pada pertemuan 7, halaman Laporan menjalankan 9 query. Kini cukup **2 request** yang berjalan paralel.

Form: tambahkan `bool _menyimpan` agar tombol **Simpan** menampilkan indikator dan tidak bisa ditekan dua kali (mencegah data ganda). Bila gagal, form **tetap terbuka**:

```
setState(() => _menyimpan = true);
try {
```

```

    if (widget.kegiatan == null) {
      await repo.tambah(k);
    } else {
      await repo.ubah(k);
    }
    if (!mounted) return;
    Navigator.pop(context);
  } catch (e) {
    // Form tetap terbuka agar isian tidak hilang.
    if (mounted) tampilkanGalat(context, e);
  } finally {
    if (mounted) setState(() => _menyimpan = false);
  }
}

```

Checkpoint E: login, tambah beberapa kegiatan, lalu periksa di Adminer: data tersimpan di tabel kegiatan dengan user_id yang benar. Login dengan akun lain di HP yang sama: daftar kosong. Matikan API (docker compose stop api): halaman menampilkan **Coba lagi**, dan menyimpan form menampilkan Snackbar tanpa menutup form. Jalankan dengan `--dart-define=MODE_DATA=lokal`: aplikasi kembali memakai SQLite tanpa perubahan kode halaman.

Bagian F: Menguji Widget dengan Repository Palsu

Karena repo adalah variabel yang dapat diganti, BerandaPage dapat diuji tanpa server. Di `test/repository_test.dart`:

```

class FakeRepository implements KegiatanRepository {
  List<Kegiatan> daftar = [];
  Object? galat;
  int jumlahPanggil = 0;

  @override
  Future<List<Kegiatan>> pada(String tanggal) async {
    jumlahPanggil++;
    if (galat != null) throw galat!;
    return daftar;
  }

  // method lain mengembalikan data sederhana; operasi tulis menaikkan versiData
}

Widget _app() => const MaterialApp(home: Scaffold(body: BerandaPage()));

testWidgets('galat -> tombol Coba lagi -> data tampil', (tester) async {
  final fake = FakeRepository()..galat = Exception('server mati');
  repo = fake;
  await tester.pumpWidget(_app());
  await tester.pumpAndSettle();
  expect(find.text('Coba lagi'), findsOneWidget);

  fake.galat = null;
  await tester.tap(find.text('Coba lagi'));
  await tester.pumpAndSettle();
  expect(find.textContaining('Belum ada kegiatan'), findsOneWidget);
});

```

Tambahkan juga test bahwa daftar dari repository tampil di KegiatanTile, bahwa `repo.hapus()` memicu halaman memuat ulang (`jumlahPanggil` bertambah), dan bahwa `ApiKegiatanRepository.tambah` menaikkan `versiData` (memakai `MockClient`).

Checkpoint F: flutter test lulus semua, termasuk test SQLite pertemuan 7 yang tetap berjalan karena DbHelper tidak dihapus.

5. Latihan Mandiri

1. Tampilkan `LinearProgressIndicator` tipis di atas daftar saat data dimuat ulang (bukan saat pertama kali).
2. Terapkan *optimistic update* pada centang: ubah tampilan lebih dulu, kirim `PATCH`, lalu kembalikan bila gagal.
3. Ukur waktu muat halaman Laporan dengan `Stopwatch` sebelum dan sesudah memakai `Future.wait`. Bandingkan hasilnya.
4. Tambahkan pencarian judul di halaman Hari Ini memakai parameter `q` dari pertemuan 9.

6. Tugas

(a) **Target tabungan di server.** Target harian saat ini tersimpan di HP, sehingga berbeda di setiap perangkat. Pindahkan ke akun:

- Backend: `PATCH /users/me` dengan skema `UserUpdate` (nama dan `target_harian` opsional, target 0–10.000.000).
- Flutter: `AuthApi.ubahTarget()` memanggil endpoint tersebut lalu memperbarui profil tersimpan. Slider di Pengaturan menyimpan ke server (bila gagal, nilai dikembalikan dan `SnackBar` tampil). Nilai `targetHarian` mengikuti `pengguna.value.targetHarian` setiap kali pengguna berganti.

(b) **Mode offline sederhana (network first).**

- Buat `CacheStore` (SQLite, berkas `tabungku_cache.db`, tabel `cache(kunci, isi, waktu)`) beserta versi memori untuk test.
- Buat `CacheClient` extends `http.BaseClient` yang membungkus `ApiClient`. Untuk request **GET** yang berhasil (200), simpan isi respons ke cache. Bila jaringan gagal (*exception/timeout*), kembalikan isi cache sebagai respons 200 dan isi `ValueNotifier<DateTime?> statusOffline` dengan waktu data tersebut. Request selain GET tidak di-cache.
- Kunci cache memuat **id pengguna + URL** agar data dua akun di satu HP tidak tertukar. Cache **dihapus saat keluar**.
- Tampilkan banner "Offline: menampilkan data tersimpan (terakhir diperbarui pukul HH:mm)" di atas `ShellPage`.
- Unit test: online menyimpan cache, offline memakai cache, offline tanpa cache melempar galat, POST tidak di-cache, akun lain tidak memakai cache, dan keluar menghapus cache.
- Kumpulkan tangkapan layar mode offline (saat `docker compose stop api`).

7. Rubrik Penilaian

Komponen	Bobot
Repository + dua implementasi; UI tanpa <code>DbHelper</code> langsung (bagian A–B)	25%
Penanganan galat, <i>pull-to-refresh</i> , form tidak tertutup saat gagal (bagian C–E)	20%
Widget test dengan <code>FakeRepository</code> (bagian F)	15%
Tugas (a): target di server	15%
Tugas (b): cache offline + banner + test	25%

8. Pertanyaan Refleksi

1. Bila suatu saat backend diganti (misalnya ke Firebase), bagian kode mana saja yang perlu diubah?
2. Mengapa repository menaikkan `versiData` hanya **setelah** request berhasil?
3. Apa risiko bila tombol Simpan bisa ditekan dua kali saat jaringan lambat?
4. Dalam strategi *network first*, kapan data yang tampil bisa usang, dan bagaimana pengguna mengetahuinya?
5. Mengapa cache harus dihapus saat pengguna keluar?

9. Troubleshooting Umum

Masalah	Solusi
Data tidak muncul setelah login	Cek header <code>Authorization</code> (bagian H pertemuan 10) dan tanggal: API memfilter tanggal hari ini sesuai zona waktu HP
<code>type 'List<Object>' is not a subtype of ... setelah Future.wait</code>	Lakukan cast per elemen: <code>hasil[0] as List<Kegiatan></code>
RefreshIndicator tidak bisa ditarik	Tambahkan <code>physics: const AlwaysScrollableScrollPhysics()</code> pada <code>ListView</code>
Halaman tidak memuat ulang setelah tambah	Repository harus menaikkan <code>versiData</code> dan halaman harus <code>addListener</code> di <code>initState</code>
Test gagal: Found 2 widgets with text	Teks muncul di beberapa tempat (misalnya ringkasan dan daftar). Batasi dengan <code>find.descendant(of: find.byType(KegiatanTile), ...)</code>
Galat <code>setState()</code> called after <code>dispose()</code>	Periksa <code>if (mounted)</code> setelah setiap <code>await</code> sebelum <code>setState</code> atau memakai <code>context</code>

10. Referensi

- Arsitektur aplikasi Flutter (lapisan data dan repository): docs.flutter.dev/app-architecture
- Pengujian widget: docs.flutter.dev/cookbook/testing/widget/introduction
- Offline-first: docs.flutter.dev/app-architecture/design-patterns/offline-first
- Modul pertemuan 4 (`FutureBuilder`) dan 7 (`versiData`)